

Sudoku Validation

Problem Statement

Validate an N x N Sudoku board ensuring no duplicates in rows, columns, and sub-boxes.

Variants

- Different board sizes and box dimensions

- Part

How to Leverage the Solution

Parameterize box dims; same row/col/box sets.

Java Solution

Below is a concise reference implementation. Full comments omitted for brevity.

```
import java.util.*;

public final class SudokuValidator {
    public static boolean isValid(char[][] board) { return isValid(board, 3, 3); }
    public static boolean isValid(char[][] board, int boxRows, int boxCols) {
        int n = board.length; for (char[] row : board) if (row.length != n) throw new IllegalArgumentException("Non-square board");
        Set<Character>[] rows = new Set[n]; Set<Character>[] cols = new Set[n];
        @SuppressWarnings("unchecked") Set<Character>[][] boxes = new Set[n / boxRows][n / boxCols];
        for (int i = 0; i < n; i++) rows[i] = new HashSet<>(); for (int j = 0; j < n; j++) cols[j] = new HashSet<>();
        for (int br = 0; br < n / boxRows; br++) for (int bc = 0; bc < n / boxCols; bc++) boxes[br][bc] = new HashSet<>();
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                char c = board[i][j]; if (c == '.') continue;
                if (rows[i].contains(c) || cols[j].contains(c) || boxes[i / boxRows][j / boxCols].contains(c))
                    return false;
                rows[i].add(c); cols[j].add(c); boxes[i / boxRows][j / boxCols].add(c);
            }
        }
        return true;
    }
}
```